

Study Case of The Ideal Weight

(462025611095 Tamam Ni'amillah Ramdhan Putra Widyana)



Pemrograman Berorientasi Objek

University of Darussalam Gontor
Faculty of Science and Technology
Informatics Engineering
2025/2026

1. Latar Belakang

Transportasi udara telah bergeser menjadi salah satu moda transportasi utama yang mendukung mobilitas global, baik untuk urusan bisnis maupun pariwisata. Seiring dengan lonjakan permintaan perjalanan, sistem pemesanan tiket dituntut untuk memberikan kecepatan, akurasi, dan efisiensi yang tinggi. Namun, realita di lapangan menunjukkan bahwa banyak agen perjalanan kecil atau sistem internal maskapai belum terdigitalisasi secara optimal.

Banyak dari mereka masih mengandalkan **proses manual atau penggunaan spreadsheet**, yang memicu berbagai masalah krusial:

- **Risiko Human Error:** Kesalahan input data penumpang atau jadwal yang fatal.
- **Visibilitas Data Rendah:** Kesulitan dalam melacak ketersediaan kursi secara *real-time*.
- **Double Booking:** Insiden di mana satu kursi dipesan oleh lebih dari satu orang karena sinkronisasi data yang buruk.
- **Kelemahan Infrastruktur Perangkat Lunak:** Sistem yang tidak terstruktur sulit untuk dipelihara (*maintainable*) dan dikembangkan (*scalable*) saat ada penambahan rute atau fitur baru.

Oleh karena itu, diperlukan sistem terkomputerisasi menggunakan **Python**. Pemilihan Python didasarkan pada sintaksnya yang bersih serta kemampuannya mendukung pengembangan sistem yang kompleks melalui paradigma **Object-Oriented Programming (OOP)** untuk memodelkan entitas dunia nyata ke dalam kode program yang logis dan modula

2. Rumusan Masalah

Berdasarkan latar belakang tersebut, poin-poin masalah yang ingin diselesaikan adalah:

1. Bagaimana membangun sistem yang mampu mengelola ketersediaan kursi secara akurat untuk mencegah *double booking*?
2. Bagaimana mengimplementasikan paradigma OOP dalam Python untuk memodelkan entitas dunia nyata (Penumpang, Pesawat, Tiket) ke dalam kode program?
3. Bagaimana menciptakan struktur kode yang modular agar mudah dipelihara (*maintainable*) dan dikembangkan di masa depan?
4. Akurasi Inventori: Bagaimana merancang sistem yang mampu mengelola ketersediaan kursi secara presisi untuk mengeliminasi risiko double booking?
5. Pemodelan Entitas: Bagaimana mengimplementasikan paradigma OOP dalam Python untuk memetakan entitas fisik (seperti Penumpang, Penerbangan, Kursi, dan Tiket) ke dalam relasi antar-kelas yang efektif
6. Keberlanjutan Kode: Bagaimana menciptakan struktur kode yang modular agar mudah dipelihara dan siap untuk pengembangan fitur skala besar di masa depan?

3. Tujuan Penelitian

Tujuan utama dari pengembangan aplikasi ini adalah:

- **Sentralisasi Data:** Menghasilkan aplikasi reservasi yang mampu mengelola jadwal penerbangan, kursi, dan data penumpang secara akurat dalam satu *database* terpusat.
- **Implementasi OOP:** Menerapkan konsep *Encapsulation*, *Inheritance*, dan *Polymorphism* untuk membangun arsitektur sistem yang kokoh.
- **Landasan Kode yang Rapi:** Menyediakan kode program yang berorientasi objek untuk memudahkan proses *debugging* dan pengembangan fitur lanjutan (seperti integrasi pembayaran).

4. Manfaat Penelitian

- **Bagi Operator/Agen:** Meningkatkan efisiensi operasional dan meminimalisir kesalahan manual dalam manajemen penerbangan.
- **Bagi Penumpang:** Memberikan pengalaman pemesanan yang lebih cepat, transparan, dan terstruktur.
- **Bagi Akademisi/Pengembang:** Menjadi referensi praktis dalam menerapkan konsep OOP pada kasus bisnis dunia nyata menggunakan Python.

5. Penjelasan Aplikasi & Implementasi OOP

A. Fitur Utama Aplikasi:

1. **Manajemen Penerbangan:** Admin dapat menambah, mengubah, atau menghapus jadwal dan rute.
2. **Pencarian & Pemesanan:** Penumpang bisa mencari jadwal berdasarkan tujuan dan memilih kursi secara langsung.
3. **Penerbitan E-Ticket:** Sistem otomatis mengubah status kursi menjadi "terpesan" dan menghasilkan kode unik.
4. **Manajemen Pengguna:** Pemisahan akses antara Admin (pengelola) dan Penumpang (pemesan).

B. Implementasi Konsep OOP:

- **Encapsulation (Enkapsulasi):** Digunakan untuk melindungi integritas data internal. Contohnya, pada kelas `Penerbangan`, atribut seperti `_daftar_penumpang_terdaftar` dibuat menjadi *protected/private*. Status kursi tidak boleh diubah sembarangan dari luar; harus melalui metode `pesan_kursi()` yang akan melakukan validasi otomatis apakah kursi tersebut benar-benar kosong sebelum statusnya diperbarui.

- **Inheritance (Pewarisan):** Meningkatkan efisiensi kode melalui *code reusability*. Dibuat kelas induk `Pengguna` (menyimpan nama, email, telepon), yang kemudian diturunkan menjadi kelas `Admin` dan `Penumpang`. `Admin` mendapatkan fitur tambahan seperti `tambah_jadwal_penerbangan()`, sementara `Penumpang` memiliki metode khusus untuk melihat `riwayat_pemesanan()`.
- **Polymorphism (Polimorfisme):** Memungkinkan metode yang sama berperilaku berbeda tergantung objeknya. Terdapat metode `hitung_harga_akhir()` di kelas dasar `KelasTiket`.
 - Pada `TiketEkonomi`, metode ini hanya menghitung harga dasar dan pajak standar.
 - Pada `TiketBisnis`, metode yang sama akan otomatis menambahkan biaya fasilitas ekstra (seperti akses *lounge* dan bagasi tambahan). Sistem utama cukup memanggil satu metode ini tanpa perlu tahu detail perhitungan di dalamnya, karena polimorfisme menangani logika tersebut secara otomatis.